Database Management Systems (DBMS)

Course Code: CSE302 | Semester: III | Academic Year: 2026-2027

Unit III: Structured Query Language (SQL) & Database Programming

TABLE OF CONTENTS

Database Management Systems (DBMS)

Course Code: CSE302 | Semester: III | Academic Year: 2026-2027

Unit III: Structured Query Language (SQL) & Database Programming

PDF Type: UNIT NOTES (Exam-Oriented & Comprehensive)

📑 TABLE OF CONTENTS

🎯 1. LEARNING OBJECTIVES

After studying this unit, students will be able to:

- Write syntactically correct SQL queries using DDL, DML, DCL, and TCL commands.
- Enforce domain, entity, and referential integrity using SQL constraints.
- Construct complex queries involving multi-table Joins and Nested Subqueries.
- Develop robust database programs using PL/SQL Stored Procedures, Functions, and Triggers.
- Analyze and optimize SQL queries for university examinations and technical interviews.

🚀 2. INTRODUCTION TO SQL

Structured Query Language (SQL) is the standard declarative programming language used to manage, query, and manipulate data stored in Relational Database Management Systems (RDBMS). Unlike procedural languages (C, Java), SQL is non-procedural: you specify *what* data you want to retrieve, not *how* to retrieve it.

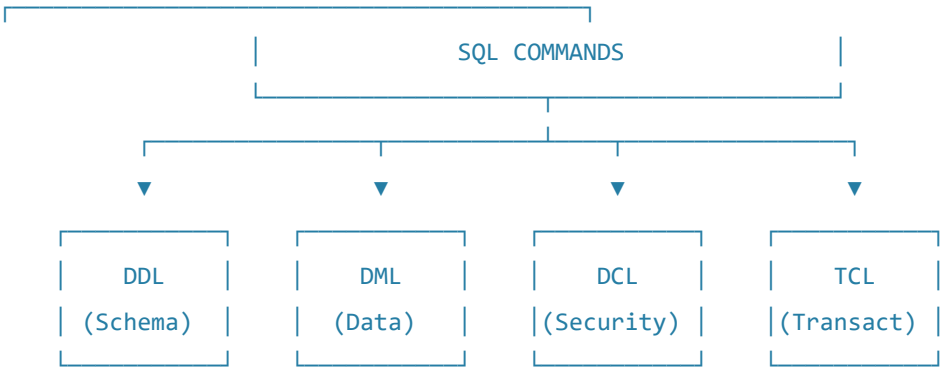
The Relational Foundations

SQL operates on relations (represented as tables). Every table consists of:

- **Columns (Attributes/Fields):** Named properties describing the data.
- **Rows (Tuples/Records):** Individual data entries.

🛠️ 3. CORE SQL DIVISIONS

SQL commands are categorized into four major sub-languages based on their operational scope.



A. Data Definition Language (DDL)

DDL commands are used to define, alter, and destroy the structural schema of the database. DDL operations are **auto-committed** (cannot be rolled back).

- `CREATE` : Defines new database structures (tables, views, indexes).

```
CREATE TABLE Student (
        student_id VARCHAR(10) PRIMARY KEY,
        name VARCHAR(50) NOT NULL,
        cgpa DECIMAL(3,2) CHECK (cgpa >= 0.00 AND cgpa <= 10.00)
    );
```

- `ALTER` : Modifies an existing schema structure.

```
ALTER TABLE Student ADD email VARCHAR(100);
    ALTER TABLE Student DROP COLUMN cgpa;
```

- `DROP` : Deletes a database object completely along with all its data.

```
DROP TABLE Student;
```

- `TRUNCATE` : Removes all rows from a table, releasing storage space, but retains the table structure.

```
TRUNCATE TABLE Student;
```

B. Data Manipulation Language (DML)

DML commands are used to insert, retrieve, update, and delete data within the database tables. DML operations **can be rolled back**.

- `INSERT` : Adds new rows to a table.

```
INSERT INTO Student VALUES ('26CSE001', 'Arjun Kumar', 'arjun@test.com');
```

- `SELECT` : Retrieves data from one or more tables.

```
SELECT name, email FROM Student WHERE student_id = '26CSE001';
```

- `UPDATE` : Modifies existing data records.

```
UPDATE Student SET email = 'arjun.k@test.com' WHERE student_id = '26CSE001';
```

- `DELETE` : Removes specific rows from a table based on a condition.

```
DELETE FROM Student WHERE student_id = '26CSE001';
```

C. Data Control Language (DCL)

DCL commands handle the security and access control of the database.

- `GRANT` : Gives specific permissions to users.

```
GRANT SELECT, INSERT ON Student TO csefaculty5;
```

- `REVOKE` : Withdraws previously granted permissions.

```
REVOKE INSERT ON Student FROM csefaculty5;
```

D. Transaction Control Language (TCL)

TCL commands manage the execution of transactions to enforce the ACID properties.

- `COMMIT` : Saves all changes made during the transaction permanently.
- `ROLLBACK` : Undoes all changes made during the transaction since the last commit.
- `SAVEPOINT` : Creates temporary checkpoints within a transaction to rollback to.

```
INSERT INTO Student VALUES ('26CSE002', 'Vijay', 'vijay@test.com');
    SAVEPOINT SP1;
    UPDATE Student SET email = 'vijay.new@test.com' WHERE student_id = '26CSE002';
    ROLLBACK TO SP1; -- Undoes the update, but keeps the insert!
    COMMIT; -- Saves the insert permanently.
```

🔒 4. SQL CONSTRAINTS

Constraints are rules enforced on data columns to preserve database integrity.

1. `NOT NULL` : Prevents null values from being inserted into a column.
2. `UNIQUE` : Guarantees all values in a column are distinct.
3. `PRIMARY KEY` : Uniquely identifies each row (`NOT NULL` + `UNIQUE`). Only one primary key is allowed per table.
4. `FOREIGN KEY` : Establishes a relationship between tables (Referential Integrity). Prevents actions that would destroy links between tables.
5. `CHECK` : Restricts the range/domain of values that can be entered.
6. `DEFAULT` : Provides a default value if none is specified.

🤝 5. SQL JOINS IN DEPTH

A **JOIN** clause is used to combine rows from two or more tables based on a related column between them.

Let's assume two sample tables for our join examples:

Student Table (Left)

| id | name | dept_id |

|---|---|---|

| S1 | Alice | D1 |

| S2 | Bob | D2 |

| S3 | Charlie | NULL |

Department Table (Right)

| dept_id | dept_name |

|---|---|---|

| D1 | CSE |

| D2 | ECE |

| D4 | Civil |

A. Inner Join

Returns only the rows that have matching values in both tables.

```
Alice (D1)  <==>  D1 (CSE)
       Bob (D2)    <==>  D2 (ECE)
```

```
SELECT Student.name, Department.dept_name
FROM Student
INNER JOIN Department ON Student.dept_id = Department.dept_id;
```

Output:

| name | dept_name |

|---|---|

| Alice | CSE |

| Bob | ECE |

B. Left Outer Join

Returns all rows from the left table, and the matched rows from the right table. If no match is found, NULL is returned for the right columns.

```
Alice (D1)  <==>  D1 (CSE)
       Bob (D2)    <==>  D2 (ECE)
       Charlie     <==>  NULL
```

```
SELECT Student.name, Department.dept_name
FROM Student
LEFT JOIN Department ON Student.dept_id = Department.dept_id;
```

Output:

| name | dept_name |

|---|---|

| Alice | CSE |

| Bob | ECE |

| Charlie | NULL |

C. Right Outer Join

Returns all rows from the right table, and the matched rows from the left table. If no match is found, NULL is returned for the left columns.

```
Alice (D1)  <═══>  D1 (CSE)
       Bob (D2)    <═══>  D2 (ECE)
       NULL        <═══>  D4 (Civil)
```

```
SELECT Student.name, Department.dept_name
FROM Student
RIGHT JOIN Department ON Student.dept_id = Department.dept_id;
```

Output:

| name | dept_name |

|---|---|

| Alice | CSE |

| Bob | ECE |

| NULL | Civil |

D. Full Outer Join

Returns all rows when there is a match in either left or right table. Unmatched records on either side are filled with NULL.

```
SELECT Student.name, Department.dept_name
FROM Student
FULL JOIN Department ON Student.dept_id = Department.dept_id;
```

Output:

| name | dept_name |

|---|---|

| Alice | CSE |

| Bob | ECE |

| Charlie | NULL |

| NULL | Civil |

E. Self Join

A regular join in which the table is joined with itself. It requires using table aliases to distinguish the table copy.

- *Use Case:* Finding manager-employee hierarchies.

```
SELECT E1.name AS Employee, E2.name AS Manager
FROM Employee E1, Employee E2
WHERE E1.manager_id = E2.employee_id;
```

🔍 6. NESTED QUERIES & SUBQUERIES

A **Subquery** (or Nested Query) is a query nested inside the `WHERE` or `HAVING` clause of another query.

A. Non-Correlated Subquery

The inner query executes once independently and returns values to the outer query.

- *Example:* Find students who score more than the class average.

```
SELECT name FROM Student
WHERE marks > (SELECT AVG(marks) FROM Student);
```

B. Correlated Subquery

The inner query references columns from the outer query. It executes repeatedly, once for each row processed by the outer query.

- *Example:* Find students who score higher than the average score of their own department.

```
SELECT S.name, S.marks, S.dept_id
FROM Student S
WHERE S.marks > (
    SELECT AVG(S2.marks)
    FROM Student S2
    WHERE S2.dept_id = S.dept_id
);
```

C. Advanced Subquery Operators

- **`IN` / `NOT IN`** : Compares a value to a list of values returned by a subquery.

- **`EXISTS` / `NOT EXISTS`** : Returns `TRUE` if the subquery returns at least one row. (More efficient than `IN` for large tables).
- **`ANY` / `ALL`** : Compares a single value against all values in the subquery result set.

```
-- Find courses where enrollment is greater than all courses in Sem 1
    SELECT title FROM Course
    WHERE enrollment > ALL (SELECT enrollment FROM Course WHERE semester = 1);
```

⚙️ 7. STORED PROCEDURES, FUNCTIONS, & TRIGGERS

Database programming moves computational logic inside the database server to reduce network latency.

PL/SQL Block Structure

PL/SQL is procedural extension of SQL. Every PL/SQL script follows this structure:

```
DECLARE
    -- Declaration of variables, constants, cursors
    v_student_name Student.name%TYPE;
BEGIN
    -- Executable SQL and procedural control logic
    SELECT name INTO v_student_name FROM Student WHERE student_id = '26CSE001';
    DBMS_OUTPUT.PUT_LINE('Name: ' || v_student_name);
EXCEPTION
    -- Error handling block
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Student not found.');
END;
```

A. Stored Procedures

A stored procedure is a schema object containing PL/SQL statements that can be invoked repeatedly by name. It **does not need to return a value** but can use `OUT` parameters.

```
CREATE OR REPLACE PROCEDURE UpdateFeeStatus(
    p_roll_number IN VARCHAR,
    p_amount IN DECIMAL
) AS
BEGIN
    UPDATE FeePayments
    SET paid_amount = paid_amount + p_amount, last_updated = CURRENT_TIMESTAMP
    WHERE roll_number = p_roll_number;

    COMMIT;
END;
```

B. Stored Functions

Functions are similar to procedures but **must return a single value** using the `RETURN` keyword. They can be used directly within SQL queries.

```
CREATE OR REPLACE FUNCTION CalculateTax(
    p_income IN DECIMAL
) RETURN DECIMAL AS
BEGIN
    RETURN p_income * 0.18; -- 18% Tax Rate
END;
```

C. Database Triggers

A trigger is a stored PL/SQL block that is automatically executed (fired) in response to a specific database event (`INSERT` , `UPDATE` , `DELETE`).

- *Types:* Row-level (fires for each modified row) vs. Statement-level (fires once per command).
- *Timing:* `BEFORE` or `AFTER` the database event.

```
-- Audit Log Trigger: Records updates to marks
CREATE OR REPLACE TRIGGER AuditStudentMarks
AFTER UPDATE OF marks ON Student
FOR EACH ROW
BEGIN
    INSERT INTO MarksAuditLog(student_id, old_marks, new_marks, change_time)
    VALUES (:OLD.student_id, :OLD.marks, :NEW.marks, SYSDATE);
END;
```

📊 8. COMPARISON & SUMMARY TABLES

DDL vs. DML

| Feature | Data Definition Language (DDL) | Data Manipulation Language (DML) |

|---|---|---|

| **Scope** | Modifies the structural database schema. | Manipulates the data stored inside schemas. |

| **Commands** | `CREATE`, `ALTER`, `DROP`, `TRUNCATE` | `SELECT`, `INSERT`, `UPDATE`, `DELETE` |

| **Commit Type** | Auto-committed (permanent instantly). | Manual commit required (can be rolled back). |

| **Speed** | Highly efficient (minimal logs). | Moderately slower (creates undo/redo logs). |

DELETE vs. TRUNCATE

| Feature | DELETE | TRUNCATE |

|---|---|---|

| **Command Type** | DML (Data Manipulation) | DDL (Data Definition) |

| **Performance** | Slower (deletes row-by-row, logs each delete). | Fast (deallocates storage pages directly). |

| **Rollback** | Possible using `ROLLBACK;`. | Not possible (Auto-committed). |

| **Filter** | Can filter using `WHERE` clause. | Cannot filter; clears the entire table. |

| **Locks** | Row-level locks acquired. | Table-level locks acquired. |

📈 9. ADVANTAGES, DISADVANTAGES, & APPLICATIONS

Advantages of SQL Databases

- **High Performance:** Optimized indexing allows fast data retrieval.
- **Data Integrity:** Restricts bad data through strict schema rules and constraint types.
- **Data Independence:** Physical structural modifications do not impact application code.
- **Standardized Security:** Strong role-based access controls using `GRANT` / `REVOKE`.

Disadvantages of SQL Databases

- **Scalability Limits:** Primarily vertically scalable (requires larger hardware resources) rather than horizontally.
- **Rigid Schemas:** Modifying existing schemas on massive live databases requires complex migration plans.
- **Object-Relational Mapping (ORM) Impedance:** Discrepancy between code objects and database relation records.

📝 10. WORKED EXAMPLES (SOLVED UNIVERSITY PROBLEMS)

Scenario Setup

Assume the following database tables:

- `Student(sid, sname, semester, gpa)`
- `Enrolled(sid, cid, grade)`
- `Course(cid, cname, credits)`

Question 1 (SQL Join):

Write an SQL query to retrieve the names of all students enrolled in the course 'Database Management Systems'.

- **Solution:**

```
SELECT S.sname
    FROM Student S
    INNER JOIN Enrolled E ON S.sid = E.sid
    INNER JOIN Course C ON E.cid = C.cid
    WHERE C.cname = 'Database Management Systems';
```

Question 2 (Nested Subquery):

Write an SQL query to find the names of students who have not enrolled in any course.

- **Solution:**

```
SELECT sname
    FROM Student
    WHERE sid NOT IN (SELECT DISTINCT sid FROM Enrolled);
```

Alternate (More optimized using `EXISTS`):

```
SELECT sname
    FROM Student S
    WHERE NOT EXISTS (
        SELECT 1
        FROM Enrolled E
        WHERE E.sid = S.sid
    );
```

💡 11. EXAM TIPS & MEMORY TECHNIQUES

> [!IMPORTANT]

> **Referential Integrity Rule:**

> If a foreign key references a primary key, you *cannot* insert a child row with a non-existent parent value, nor can you delete a parent row while active child references exist (unless using `ON DELETE CASCADE`).

> [!TIP]

> **Subquery Performance Tip:**

> Use `EXISTS` instead of `IN` when dealing with large datasets. `EXISTS` stops scanning the subquery tables the moment it finds the first matching row (short-circuit execution), whereas `IN` scans the entire subquery result set.

❓ 12. PRACTICE QUESTIONS

Short Answer Questions (2 Marks)

1. Explain the difference between SQL schemas and tables.

2. Define Referential Integrity. How is it enforced in SQL?

3. Why is a Self Join used? Give a real-world example.

4. State the difference between WHERE and HAVING clauses.

5. What are the limitations of using stored functions inside standard SELECT queries?

Long Answer Questions (10 Marks)

1. Explain DDL, DML, and TCL commands in detail with code examples for each.

2. Analyze Joins in SQL. Discuss the implementation and output differences of Inner Join, Left Join, Right Join, and Full Join with appropriate schema examples.

3. Design a PL/SQL trigger that automatically fires to log deletion records of a 'Student' table into a backup table. Provide the DDL for the tables and the full trigger script.

🎓 13. HIGH-YIELD VIVA QUESTIONS

1. **Q: What is a NULL value in SQL?**

- *A:* NULL represents a missing, unknown, or inapplicable data value. It is not equivalent to zero or a space.

2. **Q: What is the difference between primary keys and unique keys?**

- *A:* A table can have only one Primary Key and it cannot contain NULLs. A table can have multiple Unique Keys, and they can accept one or more NULL values.

3. **Q: Can we rollback a TRUNCATE command?**

- *A:* No, TRUNCATE is a DDL command and is auto-committed, so it cannot be rolled back in standard relational databases.

4. **Q: What is the difference between JOIN and UNION?**

- *A:* JOIN combines columns horizontally from different tables based on match keys; UNION combines rows vertically from similar query sets (must have identical column counts and data types).

5. **Q: What are cursors in PL/SQL?**

- *A:* A cursor is a pointer to a private SQL memory area containing execution context information for SELECT or DML commands.

📌 14. QUICK REVISION SUMMARY

- **SQL is Declarative:** Focuses on *what* rather than *how*.
- **Auto-commit:** DDL operations commit immediately.
- **Triggers:** Fire automatically based on database actions (`BEFORE` / `AFTER` DML).
- **Views:** Virtual tables generated dynamically via stored query expressions.
- **Subqueries:** Queries nested inside clauses to partition execution results.